

Programming Assignment 1 – Algorithm Analysis and correctness

Objective: In real-world coding, you may use an LLM or existing codebase as a starting point, but you must still prove correctness, analyze complexity, and improve the solution. This assignment challenges you to do exactly that with the classic Two-Sum problem.

Problem Statement:

Given an array of integers and a target integer, return true if there exist two distinct numbers in the array whose sum is equal to the target.

You will examine two natural approaches:

- Naive Approach ($O(n^2)$) – check all pairs.
- Optimized Approach ($O(n)$) – use a hash map to store previously seen numbers.

You write your own code as a starting point (or generate your own via your choice of LLM).

Also, if you use LLM, give the five prompts that led you to the solution.

Part A – Theoretical Analysis

Time Complexity

- Derive the runtime for the naive solution (best, worst, average).
- Derive the runtime for the optimized solution.
- Compare space usage of both. Need not give the complexity, just talk about the storage required.

Loop Invariants

- State an invariant for the inner loop of the naive solution.
- State an invariant for the optimized solution (hint: what does seen represent at each step?).

Proof of Correctness

- Use the Initialization–Maintenance–Termination framework with your invariants to show correctness for both algorithms.

Comparison & Discussion

- In what situations might the naive version still be acceptable?
- How does space-time tradeoff affect which version you'd pick in practice?

Part B – Vibe Coding Tasks

Interactively trace and analyze the solutions:

1. Trace Naive Algorithm (by hand)
 - a. Run twoSumNaive on input [2, 7, 11, 15] with target = 9.
 - b. Record the comparisons made step by step.
 - c. At what point does the algorithm return true?
2. Trace Optimized Algorithm (by hand)
 - a. Run twoSumOptimized on the same input.
 - b. Record the state of the seen set after each iteration.
 - c. Why does the algorithm terminate as soon as it finds the complement?
3. LLM Pair Programming
 - a. Ask an LLM to implement Two-Sum in Java and mention the five prompts that led you to the solution. Save the code it produces.
 - b. Run it on test cases. Does it handle duplicates correctly? Empty arrays? Negative numbers?
 - c. Critical Task: Use invariants and complexity analysis to evaluate whether the LLM's solution is truly correct.
 - d. Modify one version to also return the indices of the two numbers (not just true/false).
 - e. Ask an LLM to help, then critique and fix its output if necessary.

Deliverables

1. Report (PDF) containing:
 - a. Time and space complexity analysis.
 - b. Loop invariants and correctness proofs.
 - c. Trace outputs for both algorithms.
 - d. LLM output and your evaluation of its correctness.
 - e. Reflection: Would you trust the LLM's solution in an interview? Why or why not?
2. Code (.java) file with:
 - a. Naive and optimized implementations.
 - b. Your modified version (indices).
 - c. Any LLM-generated version you tested.